Trabajo

Tarea Implementación de la estructura de una base de datos.

Estudiantes

Mariana López Díaz.

Sara Ximena Machado.

Asignatura

Bases de Datos.

Docente

ORALIA CORTES GRAJALES

Institucion Universitaria Pascual Bravo

Facultad de Ingeniería

Tecnología de desarrollo de Software

Medellín

2026

El proyecto Red Social Estudiantil Pascualina consiste en diseñar e implementar una base de datos para una plataforma que permita a los estudiantes compartir publicaciones, comentar, identificar intereses y habilidades, crear grupos de estudio, participar en eventos y seguir a otros estudiantes.

El trabajo se desarrolló en tres etapas: diseño del modelo entidad-relación, construcción del modelo lógico y creación del modelo físico mediante PostgreSQL. Finalmente, se realizaron pruebas para comprobar la creación de las tablas, las relaciones y la recuperación de los datos mediante un archivo SQL.

Identificación del SGBD y justificación de su elección

El sistema gestor de bases de datos (SGBD) seleccionado fue **PostgreSQL 17**, utilizando **pgAdmin 4** como herramienta gráfica de administración.

PostgreSQL es un sistema gestor de bases de datos relacional que permite organizar la información en tablas, establecer relaciones mediante claves primarias y foráneas, definir restricciones y ejecutar consultas SQL.

Ventajas de PostgreSQL

- Es gratuito y de código abierto.
- Permite establecer claves primarias, claves foráneas y restricciones para mantener la integridad de los datos.
- Admite diferentes tipos de datos, como INTEGER, VARCHAR, TEXT, DATE, TIME y TIMESTAMP.
- Permite generar identificadores automáticamente mediante columnas IDENTITY.
- Facilita la creación de copias de seguridad y la recuperación de bases de datos.
- pgAdmin 4 ofrece una interfaz gráfica para crear bases de datos, ejecutar consultas y revisar las tablas.

Comparación con otras opciones

SGBD	Ventajas	Desventajas
PostgreSQL	Gratuito y con buenas herramientas para manejar relaciones y restricciones.	Requiere familiarizarse con su instalación y administración.
MySQL	Popular, gratuito en su edición comunitaria y ampliamente utilizado en aplicaciones web.	Algunas características y comportamientos difieren de

		PostgreSQL, por lo que habría que adaptar ciertos comandos.
SQL Server	Ofrece herramientas de administración y soluciones para entornos empresariales.	Sus opciones de licenciamiento y edición pueden implicar costos o restricciones adicionales.
SQLite	Es sencillo de instalar y no requiere un servidor independiente.	Resulta menos adecuado para practicar una administración basada en servidor como la realizada en esta actividad.

Se eligió PostgreSQL porque cumple con los requerimientos del proyecto y permite implementar las tablas, las claves y las restricciones necesarias. Además, su integración con pgAdmin 4 facilitó el proceso de creación, consulta y verificación de la base de datos.

Descripción del modelo físico

El modelo físico corresponde a la implementación del diseño lógico en un SGBD específico. En esta etapa se definieron las tablas, los campos, los tipos de datos, las claves primarias, las claves foráneas y las restricciones.

La base de datos se creó con el nombre red_social_pascualina y quedó organizada en 12 tablas:

1. estudiante
2. publicacion
3. comentario
4. interes
5. estudiante_interes
6. habilidad
7. estudiante_habilidad
8. grupo
9. membresia
10. seguimiento
11. evento
12. participacion_evento

Las tablas se relacionan para evitar la repetición innecesaria de información. Por ejemplo, los intereses y las habilidades se almacenan en tablas independientes y se vinculan con los estudiantes mediante tablas asociativas.

Transformación del modelo lógico al modelo físico

El proceso se realizó de la siguiente manera:

1. Se revisó el diseño elaborado en las actividades anteriores y se identificaron las entidades, sus atributos y sus relaciones.
2. Cada entidad se convirtió en una tabla de PostgreSQL.
3. Se asignaron tipos de datos según la información que almacena cada campo.
4. Se definieron las claves primarias para identificar los registros.
5. Se implementaron las claves foráneas para relacionar las tablas.
6. Se agregaron restricciones como NOT NULL, UNIQUE y CHECK cuando correspondía.
7. Se incluyeron datos de prueba para comprobar el funcionamiento.
8. Se generó un archivo SQL y se probó su restauración en una base de datos de prueba.

Justificación de los tipos de datos

Los tipos de datos se seleccionaron de acuerdo con la información que debe almacenar cada campo:

- **INTEGER:** se utiliza para los identificadores y las referencias entre tablas. Es suficiente para el volumen previsto del proyecto y ocupa menos espacio que BIGINT.
- **VARCHAR:** se utiliza en nombres, apellidos, correos, lugares y categorías porque permite establecer una longitud máxima.
- **TEXT:** se utiliza en descripciones, publicaciones y comentarios porque su contenido puede tener una extensión variable.
- **DATE:** almacena fechas sin incluir la hora, como el nacimiento de un estudiante o la fecha de un evento.
- **TIME:** almacena la hora de inicio de un evento.
- **TIMESTAMP:** almacena fecha y hora para registrar publicaciones, comentarios, inscripciones y fechas de creación.
- **IDENTITY:** permite que PostgreSQL genere automáticamente identificadores numéricos.
- **DEFAULT CURRENT_TIMESTAMP:** asigna la fecha y hora actuales cuando no se proporciona un valor para determinados campos.

Se utilizaron longitudes específicas en VARCHAR para establecer límites razonables. Por ejemplo, correo tiene una longitud máxima de 150 caracteres y tipo en publicacion tiene una longitud máxima de 50 caracteres.

Descripción de las tablas implementadas

En las siguientes tablas se presenta el inventario de campos, sus tipos de datos y las claves primarias. Cuando un campo no tiene una longitud explícita, se indica el tipo correspondiente de PostgreSQL.

Tabla estudiante

Almacena la información básica de los estudiantes registrados en la plataforma.

Campo	Tipo de dato	Descripción
id_estudiante	INTEGER	Identificador automático del estudiante.
nombre	VARCHAR(100)	Nombre del estudiante.
apellido	VARCHAR(100)	Apellido del estudiante.
correo	VARCHAR(150)	Correo electrónico, obligatorio y único.
fecha_nacimiento	DATE	Fecha de nacimiento; puede quedar vacía.

PK: id_estudiante.

Se eligió un identificador numérico porque permite distinguir cada estudiante sin depender de datos personales que pueden cambiar. El correo tiene una restricción UNIQUE para evitar registros con el mismo correo.

Tabla publicacion

Almacena las publicaciones creadas por los estudiantes.

Campo	Tipo de dato	Descripción
id_publicacion	INTEGER	Identificador automático de la publicación.
id_estudiante	INTEGER	Estudiante que creó la publicación.
contenido	TEXT	Texto de la publicación.
fecha_publicacion	TIMESTAMP	Fecha y hora de publicación.
tipo	VARCHAR(50)	Categoría de la publicación.

PK: id_publicacion.

El contenido se definió como TEXT para permitir publicaciones de extensión variable. El campo tipo permite clasificar publicaciones, por ejemplo, como preguntas o recursos. La fecha tiene como valor predeterminado la fecha y hora actuales.

Tabla comentario

Almacena los comentarios realizados por los estudiantes en las publicaciones.

Campo	Tipo de dato	Descripción
id_comentario	INTEGER	Identificador automático del comentario.
id_publicacion	INTEGER	Publicación comentada.
id_estudiante	INTEGER	Estudiante que escribe el comentario.
contenido	TEXT	Texto del comentario.
fecha_comentario	TIMESTAMP	Fecha y hora del comentario.

PK: id_comentario.

Se utiliza un identificador propio para distinguir cada comentario. Los campos id_publicacion e id_estudiante permiten identificar la publicación a la que pertenece y quién lo realizó.

Tabla interes

Almacena los intereses que pueden asociarse con los estudiantes.

Campo	Tipo de dato	Descripción
id_interes	INTEGER	Identificador automático del interés.
nombre	VARCHAR(100)	Nombre del interés.
descripcion	TEXT	Descripción del interés.

PK: id_interes.

El identificador permite referenciar cada interés sin repetir su descripción en todos los registros de estudiantes.

Tabla estudiante_interes

Relaciona a los estudiantes con sus intereses.

Campo	Tipo de dato	Descripción
id_estudiante	INTEGER	Identificador del estudiante.
id_interes	INTEGER	Identificador del interés.

PK compuesta: (id_estudiante, id_interes).

La clave primaria está formada por los dos campos porque cada combinación identifica una asociación entre un estudiante y un interés. Esto impide registrar dos veces la misma asociación.

Tabla habilidad

Almacena las habilidades que pueden registrar los estudiantes.

Campo	Tipo de dato	Descripción
id_habilidad	INTEGER	Identificador automático de la habilidad.
nombre	VARCHAR(100)	Nombre de la habilidad.
descripcion	TEXT	Descripción de la habilidad.
nivel	VARCHAR(50)	Nivel de dominio registrado.

PK: id_habilidad.

Se utilizan VARCHAR para el nombre y el nivel porque son datos de texto con una longitud máxima definida. La descripción se almacena en TEXT para permitir contenido variable.

Tabla estudiante_habilidad

Relaciona a los estudiantes con las habilidades que poseen.

Campo	Tipo de dato	Descripción
id_estudiante	INTEGER	Identificador del estudiante.
id_habilidad	INTEGER	Identificador de la habilidad.

PK compuesta: (id_estudiante, id_habilidad).

La combinación de ambos campos evita duplicar la misma habilidad para un mismo estudiante. Cada campo también funciona como clave foránea.

Tabla grupo

Almacena los grupos académicos o sociales de la plataforma.

Campo	Tipo de dato	Descripción
id_grupo	INTEGER	Identificador automático del grupo.
nombre	VARCHAR(100)	Nombre del grupo.
descripcion	TEXT	Descripción del grupo.
tipo	VARCHAR(50)	Clasificación del grupo.
fecha_creacion	TIMESTAMP	Fecha y hora de creación.

PK: id_grupo.

El identificador permite distinguir cada grupo. El tipo facilita clasificarlo, por ejemplo, como académico, y la fecha de creación se asigna automáticamente si no se proporciona.

Tabla membresia

Registra la pertenencia de los estudiantes a los grupos.

Campo	Tipo de dato	Descripción

id_estudiante	INTEGER	Estudiante que pertenece al grupo.
id_grupo	INTEGER	Grupo al que pertenece.
fecha_ingreso	TIMESTAMP	Fecha y hora de ingreso.
id_membresia	INTEGER	Identificador automático de la membresía.
rol	VARCHAR(50)	Rol del estudiante dentro del grupo.

PK: id_membresia.

Se utiliza un identificador propio para cada membresía. Además, se definió una restricción UNIQUE sobre (id_estudiante, id_grupo) para evitar que un estudiante tenga registrada más de una membresía en el mismo grupo.

Tabla seguimiento

Registra las relaciones de seguimiento entre estudiantes.

Campo	Tipo de dato	Descripción
id_seguidor	INTEGER	Estudiante que sigue a otro.
id_seguido	INTEGER	Estudiante que recibe el seguimiento.
id_seguimiento	INTEGER	Identificador automático del seguimiento.
fecha_seguimiento	TIMESTAMP	Fecha y hora del seguimiento; puede quedar vacía.

PK: id_seguimiento.

Se eligió un identificador propio para distinguir cada seguimiento. También se estableció una restricción UNIQUE para la pareja (id_seguidor, id_seguido) y una restricción CHECK para impedir que un estudiante se siga a sí mismo.

Tabla evento

Almacena los eventos académicos o sociales de la plataforma.

Campo	Tipo de dato	Descripción
id_evento	INTEGER	Identificador automático del evento.
nombre	VARCHAR(150)	Nombre del evento.
descripcion	TEXT	Descripción del evento.
fecha	DATE	Fecha del evento.
hora	TIME	Hora del evento.
lugar	VARCHAR(200)	Lugar de realización.

PK: id_evento.

Se separaron la fecha y la hora para permitir consultar cada dato por separado. El nombre tiene una longitud máxima de 150 caracteres y el lugar de 200, mientras que la descripción admite una extensión variable.

Tabla participacion_evento

Registra la inscripción de los estudiantes a los eventos.

Campo	Tipo de dato	Descripción
id_estudiante	INTEGER	Estudiante inscrito.
id_evento	INTEGER	Evento al que se inscribe.
fecha_inscripcion	TIMESTAMP	Fecha y hora de inscripción.

PK compuesta: (id_estudiante, id_evento).

La clave primaria compuesta evita registrar más de una inscripción del mismo estudiante al mismo evento. La fecha de inscripción se asigna automáticamente cuando no se indica un valor.

Implementación de las relaciones entre tablas

Las claves foráneas permiten relacionar las tablas y asegurar que las referencias correspondan a registros existentes. En las relaciones de este proyecto, las acciones referenciales de actualización no se escribieron expresamente en el script; por tanto, PostgreSQL aplica el comportamiento predeterminado ON UPDATE NO ACTION. Para las eliminaciones, se configuró ON DELETE CASCADE en varias relaciones y se dejó el comportamiento predeterminado en otras.

Relaciones implementadas

Tabla padre	Tabla hija	Campo de la clave foránea	ON DELETE	ON UPDATE
estudiante	publicacion	publicacion.id_estudiante	NO ACTION	NO ACTION
estudiante	comentario	comentario.id_estudiante	NO ACTION	NO ACTION
publicacion	comentario	comentario.id_publicacion	CASCADE	NO ACTION
estudiante	estudiante_intereses	estudiante_interes.id_estudiante	CASCADE	NO ACTION
interes	estudiante_intereses	estudiante_interes.id_interes	CASCADE	NO ACTION
estudiante	estudiante_habilidad	estudiante_habilidad.id_estudiante	CASCADE	NO ACTION
habilidad	estudiante_habilidad	estudiante_habilidad.id_habilidad	CASCADE	NO ACTION
estudiante	membresia	membresia.id_estudiante	CASCADE	NO ACTION
grupo	membresia	membresia.id_grupo	CASCADE	NO ACTION
estudiante	seguimiento	seguimiento.id_seguidor	CASCADE	NO ACTION
estudiante	seguimiento	seguimiento.id_seguido	CASCADE	NO ACTION

estudiante	participacion_evento	participacion_evento.id_estudiante	CASCADE	NO ACTION
evento	participacion_evento	participacion_evento.id_evento	CASCADE	NO ACTION

Justificación de las acciones referenciales

ON DELETE CASCADE: se configuró para eliminar automáticamente los registros dependientes cuando se elimina el registro padre. Por ejemplo, si se elimina una publicación, también se eliminan sus comentarios. Lo mismo sucede con las asociaciones de intereses, habilidades, membresías, seguimientos y participaciones en eventos.

ON DELETE NO ACTION: se mantiene en las relaciones de estudiante con publicacion y comentario. Esto evita eliminar un estudiante que todavía tenga publicaciones o comentarios relacionados, salvo que se eliminen primero esos registros dependientes.

ON UPDATE NO ACTION: es el comportamiento predeterminado de PostgreSQL cuando no se especifica otra acción. Impide modificar una clave referenciada si el cambio deja referencias dependientes incompatibles. En este proyecto, los identificadores se utilizan como claves estables, por lo que no se espera que deban modificarse.

Las claves foráneas se definieron para mantener la integridad referencial y evitar que existan publicaciones, comentarios, membresías o inscripciones que hagan referencia a estudiantes, grupos o eventos inexistentes.

Descripción del script DDL

El archivo entregado se llama red_social_pascualina.sql. Es un volcado SQL generado con PostgreSQL 17.11 mediante la herramienta pg_dump. Contiene tanto instrucciones para reconstruir la estructura como los datos de prueba y las restricciones de la base de datos.

Su estructura principal es la siguiente:

1. **Configuración inicial:** contiene instrucciones para establecer parámetros de ejecución, codificación y otras opciones de PostgreSQL.
2. **Creación de tablas:** incluye instrucciones CREATE TABLE para las 12 tablas.
3. **Generación de identificadores:** define columnas IDENTITY y sus secuencias asociadas.

4. **Inserción de datos:** contiene instrucciones COPY con los registros de prueba.
5. **Definición de claves primarias:** utiliza ALTER TABLE para establecer las PK.
6. **Definición de restricciones:** agrega restricciones UNIQUE, CHECK y claves foráneas.
7. **Ajuste de secuencias:** utiliza instrucciones setval para que los identificadores automáticos continúen desde el valor correspondiente después de restaurar los datos.

El archivo es un volcado SQL de la base de datos implementada; por ello, incluye más elementos que un script DDL escrito únicamente con sentencias CREATE TABLE. Esto permite recuperar tanto la estructura como los datos que estaban registrados.

7. Paso a paso de la implementación

Paso 1. Revisión del diseño

Se revisó el modelo lógico elaborado en las actividades anteriores para identificar las entidades, los atributos y las relaciones que debían implementarse.

Paso 2. Selección e instalación de las herramientas

Se utilizó PostgreSQL 17 como SGBD y pgAdmin 4 como interfaz gráfica para administrar el servidor y ejecutar consultas SQL.

Paso 3. Creación de la base de datos

Se creó la base de datos red_social_pascualina desde pgAdmin 4.

Paso 4. Creación de las tablas

Se ejecutaron las sentencias SQL necesarias para crear las 12 tablas y definir sus campos con los tipos de datos correspondientes.

Paso 5. Definición de las claves y restricciones

Se configuraron las claves primarias, las claves foráneas y las restricciones de unicidad y validación. Durante esta etapa se revisaron algunos campos que debían estar presentes, como tipo en publicacion, id_membresia y fecha_ingreso en membresia, y id_seguimiento en seguimiento.

Paso 6. Inserción de datos de prueba

Se agregaron registros de ejemplo para comprobar el funcionamiento de las tablas y las relaciones. Entre ellos se incluyeron estudiantes, publicaciones, comentarios, intereses, habilidades, grupos, membresías, seguimientos y eventos.

Paso 7. Verificación de la implementación

Se consultaron las tablas desde pgAdmin 4 y se ejecutaron consultas para comprobar que los registros estuvieran relacionados correctamente. También se verificó que las 12 tablas aparecieran en el esquema public.

Paso 8. Generación del archivo SQL

Se generó una copia de seguridad en formato SQL de texto mediante pgAdmin. El archivo contiene la estructura, las restricciones y los datos registrados.

Paso 9. Prueba de restauración

Para comprobar que el archivo podría utilizarse de forma independiente, se creó una base de datos de prueba llamada red_social_prueba y se restauró el archivo SQL. Después se verificó que las 12 tablas estuvieran presentes y que los registros se hubieran recuperado.

Dificultades encontradas y soluciones

Durante el proceso se presentaron algunas dificultades técnicas:

- **Instalación y apertura de las herramientas:** inicialmente se encontró PostgreSQL instalado, pero fue necesario instalar y abrir pgAdmin 4 para facilitar la administración.
- **Error de tabla existente:** al ejecutar nuevamente sentencias de creación aparecieron errores como “la relación ya existe”. Se identificó que algunas tablas ya habían sido creadas y que no era necesario volver a crearlas sin revisar el estado de la base de datos.
- **Inserción de identificadores automáticos:** se presentó un error relacionado con columnas GENERATED ALWAYS AS IDENTITY. Esto permitió comprender que PostgreSQL controla la generación de determinados identificadores automáticamente.
- **Restauración y verificación:** al probar el archivo SQL fue necesario revisar la conexión y confirmar que la restauración se realizara en la base de datos de prueba correcta.

Estas dificultades ayudaron a comprender mejor el funcionamiento de PostgreSQL, la administración de tablas, las restricciones y el proceso de copia de seguridad y recuperación.

Conclusión Mariana López:

Durante esta actividad aprendí cómo implementar una base de datos utilizando PostgreSQL, pasando del diseño lógico a la creación de tablas, campos y relaciones. Al principio tuve algunas dificultades con los errores que aparecían al ejecutar las instrucciones SQL, pero al revisar la estructura y realizar pruebas pude comprender mejor cómo funciona el sistema. También aprendí la importancia de las claves primarias y foráneas para mantener la información organizada y relacionada correctamente.

Conclusión Sara Machado:

La realización de este trabajo me permitió fortalecer mis conocimientos sobre bases de datos y comprender la importancia de elegir correctamente los tipos de datos y establecer restricciones. Una de las partes más importantes fue generar el archivo SQL y comprobar que la base de datos pudiera restaurarse en otra base de prueba. Esto me enseñó que no basta con crear las tablas, sino que también es necesario verificar que todo funcione correctamente y conservar una copia del trabajo realizado.

.